
Лабораторная работа №4

по курсу «Информатика (полиморфные абстрактные типы данных)» (2 семестр)

Варианты заданий

Постановка задачи

Исходными для данной работы являются реализованные в предыдущем семестре АТД Sequence, а также бинарное дерево поиска и бинарная куча. Выполнить реализацию АТД LazySequence<T>. Выполнить реализацию семейства АТД Stream<T>. С помощью этих структур реализовать одну или несколько тематических задач. Все алгоритмы покрыть модульными тестами. Оснастить реализацию пользовательским интерфейсом.

Минимальные требования к программе. .

Требования к процедуре тестирования. Должна быть предусмотрена возможность тестирования в ручном режиме, когда все тестовые данные вводятся с помощью клавиатуры. Это касается всех типов пользовательских интерфейсов, предусмотренных в реализации. Должны быть заранее заготовлены тестовые данные: коллекции, наборы правил и т.п. Должна быть предусмотрена возможность тестирования на предельных значениях: для лент машин Тьюринга и Маркова – десятки и сотни тысяч, миллионы символов; для потоков – миллионы элементов, файлы размером в гигабайты и десятки гигабайт. Должна быть предусмотрена возможность тестирования в автоматизированном режиме, путем прогона набора тестовых коллекций – заготовленных заранее или/и генерируемых на лету.

1. АТД LazySequence

АТД LazySequence реализует концепцию ленивого списка (или, если использовать введенную ранее терминологию, ленивой последовательности), т.е. ситуации, когда существует какое-то порождающее правило, по которому любой i -й элемент можно вычислить, возможно, на основе вычисленных ранее, но при этом последовательность в целом бесконечна в интуиционистском смысле¹. Типичные примеры: последовательность факториалов $\{n! | n = \overline{1, \infty}\}$, последовательности Фибоначчи.

С точки зрения классификации структур данных, ленивый список (будем использовать это обозначение как более удобный синоним для термина «ленивая последовательность») — это линейная структура с последовательным доступом. Поскольку в общем случае вычисление значения i -го элемента требует вычисления значений всех предшествующих элементов. Однако если известно, что за время работы программы потребуется не более, чем n первых элементов последовательности, то среднее время получения элемента по индексу будет асимптотически стремиться к $O(1)$ за счет мемоизации уже вычисленных значений. Заметим, что в рассматриваемом контексте (ленивые) списки могут быть как конечными, так и

¹ На практике это означает, что не требуется хранить всю бесконечность целиком и сразу. Достаточно понимания, что потенциально мы можем, за бесконечное время и при наличии бесконечного объема памяти, это сделать.

бесконечными структурами. Простейший пример, когда ленивая последовательность является конечной – это ситуация, когда элементов в ней много, а хранятся они, например, в файле на диске или получаются по сети – т.е. материализация последовательности целиком является либо невозможной, либо нежелательной, по каким-либо причинам.

Особенность ленивых списков по сравнению с обычными линейными структурами состоит в деталях реализации ряда операций, таких как все виды вставки (в начало – `prepend`, в конец – `append`, в середину – `insert`), удаление и некоторые другие. На первый взгляд может показаться, что эти операции вообще не должны быть определены на ленивых последовательностях, и тем более контринтуитивным может показаться, что эти операции работают в целом почти так же, как и для обычных последовательностей, особенно если учесть, что ленивая последовательность не обязана быть бесконечной. Отличие заключается в деталях реализации, в первую очередь, бесконечных ленивых списков. В общих чертах, идея состоит в том, что результирующая последовательность не материализуется как набора элементов; вместо этого (а) сохраняется ссылка (в широком смысле этого слова) на исходную последовательность, (б) в отдельном списке (а лучше, в очереди с приоритетами) запоминается, в какой позиции нужно внести изменение, и какое это изменение – удаление одного или нескольких элементов, добавление элементов или (бесконечного) списка, и т.п.

Ниже приводится описание примерной внешней спецификации ленивой последовательности, при этом можно заметить, что интерфейс `LazySequence` мало отличается от `Sequence`. Этого можно добиться, правильно выделив ответственности, декомпозировав объект и введя необходимые абстракции. Вопросы, которые послужат отправной точкой для дальнейших рассуждений, следующие:

- 1) Как добавлять элементы в ленивую последовательность? В частности, что значит вставить элемент в конец бесконечной ленивой последовательности? А также, что значит вставить элемент (или целую последовательность, в т.ч., возможно, ленивую), в произвольную позицию, заданную индексом?
- 2) Как обращаться к элементам, в т.ч. как их перечислить. Это также включает ответ на вопрос, каким образом и в какой момент материализуется очередной элемент последовательности и какой конкретно компонент за это отвечает.

На первый из этих вопросов ответ уже частично дан выше: мы не порождаем результирующую последовательность, а как бы запоминаем, что нужно сделать с исходной, чтобы получить желаемых результат. А ответ на второй вопрос подсказывает способ упростить решение обеих проблем: выделить ответственность «порождение очередного элемента последовательности, заданной процедурно», а объект, реализующий эту ответственность, назовем просто – генератором. Этот объект инкапсулирует правило порождения очередного элемента, которое может быть как рекуррентным, так и нет. Основным методом у такого объекта – `GetNext()` – возвращает очередной элемент. Генератор в плане интерфейса напоминает итератор, но в отличие от последнего (а) существует строго в единственном экземпляре для каждого списка, (б) не доступен извне списка, (в) может менять объект, которому принадлежит.

Следует обратить внимание, что:

- (а) с одной стороны, ленивая последовательность является неизменяемым объектом в том плане, что в интуиционистском смысле это константа, т.е. сама последовательность в принципе изменена быть не может, просто означает (материализуется) «не сразу», а «постепенно», а ее бесконечная природа проявляется в том, что процесс перечисления элементов может продолжаться потенциально бесконечно;

- (b) с другой стороны, это не обычный неизменяемый объект, какие мы прежде рассматривали, в частности, тут практически нет константных методов, т.к., например, получения элемента по индексу потенциально требует выполнение некоторых вычислений и их мемоизацию, а значит и изменение того, что в мире императивного программирования принято называть «внутренним состоянием объекта».

Еще раз напомним, что произвольно взятая ленивая последовательность не обязательно бесконечна; однако концепция ленивых структур данных возникла именно в связи с обсуждением возможностей представления бесконечных объектов.

template <class T> class LazySequence	
Создание объекта	
LazySequence ();	Создать пустой список
LazySequence (T* items, int count); LazySequence (Sequence<T>* seq);	Копировать элементы из переданного массива
LazySequence (T(*) (Sequence<T>*), Sequence<T>) LazySequence (std::function<T (Sequence<T>*)>, Sequence<T>*)	Задание (рекуррентного) порождающего правила, использующего k предшествующих элементов для порождения $(n + 1)$ -го, где $k \leq n$, а также списка первых k элементов
LazySequence (LazySequence <T> & list const);	Копирующий конструктор
Декомпозиция	
T GetFirst(); Может выбрасывать исключения: — IndexOutOfRangeException (если список пуст)	Получить первый элемент в списке
T GetLast(); Может выбрасывать исключения: — IndexOutOfRangeException (если список пуст) Вычисление может не завершиться ввиду бесконечности последовательности	Получить последний элемент в списке
T Get(int index); Может выбрасывать исключения: — IndexOutOfRangeException (если индекс отрицательный или больше/равен числу элементов) Вычисление может не завершиться ввиду бесконечности последовательности	Получить элемент по индексу.

<code>LazySequence <T>* GetSubsequence(int startIndex, int endIndex);</code> Может выбрасывать исключения: — <code>IndexOutOfRangeException</code> (если хотя бы один из индексов отрицательный или больше/равен числу элементов)	Получить список из всех элементов, начиная с <code>startIndex</code> и заканчивая <code>endIndex</code> .
<code>Cardinal GetLength() const;</code>	Получить длину списка. Тип <code>Cardinal</code> предназначен для представления множества, включающего обычные натуральные числа и первое трансфинитное число.
<code>size_t GetMaterializedCount() const;</code>	Получить количество материализованных элементов
Операции	
<code>Sequence<T>* Append(T item);</code>	Добавляет элемент в конец списка
<code>Sequence<T>* Prepend(T item);</code>	Добавляет элемент в начало списка
<code>LazySequence <T>* InsertAt(T item, int index);</code> Может выбрасывать исключения: — <code>IndexOutOfRangeException</code> (если индекс отрицательный или больше/равен числу элементов)	Вставляет элемент в заданную позицию
<code>LazySequence <T>* Concat(LazySequence <T> *list);</code>	Сцепляет два списка. Очевидно, если первый список бесконечный, то перечисление (например, итератор) дойдет до элементов второго списка только в бесконечности
<code>LazySequence<T2>* Map(T2 (*)(T));</code> <code>T2 Reduce(T2 (*)(T2,T));</code> <code>LazySequence<T>* Where(bool (*)(T));</code> <code>LazySequence<T>* Zip(Sequence<T>* seq);</code> ... <code>LazySequence<Tuple<n,T></code>	Реализация многих операций из группы <code>map-reduce</code> ничем принципиально не отличается от реализации этих операций для обычных последовательностей, поскольку использует либо получение элемента по индексу, либо перечисление элементов (т.е. итерирование), что и позволяет абстрагироваться от деталей, связанных с ленивостью и мемоизацией.

До сих пор мы описывали последовательность, «бесконечную в одну сторону». Однако такие конструкции, как лента в машине Тьюринга или Маркова, подсказывают, что начальной позицией может оказаться не положение с краю, а «в середине». Наиболее простым

решением видится применение целочисленных, в т.ч. отрицательных, индексов, а не натуральных, как обычно.

Ниже приведена примерная внешняя спецификация типа Generator. Как видно, есть много общего с итератором.

template <class T> class Generator	
Создание объекта	
Generator (LazySequence<T>*, std::function<T(Sequence<T>*)>);	Создать генератор на основе (рекуррентного) правила. Передается ссылка на объект-хозяин генератора. Генератор должен иметь доступ к внутреннему содержимому своего хозяина.
Generator (LazySequence<T>*, size_t index, Sequence<T>*,);	Создать генератор, который применит к последовательности в позиции index требуемое изменение: добавление или удаление элемента. Следует учесть, что если какие-то элементы исходной последовательности заменены, последующие должны генерироваться так, как если бы они не были заменены; это вытекает из установленного ранее принципа неизменности ленивых последовательностей. Для этого не обязательно хранить все «исходные» элементы. Достаточно помнить лишь столько, сколько необходимо, чтобы породить очередной элемент. Для этого удобно использовать очередь фиксированной длины ² .
Декомпозиция	
T GetNext(); Может выбрасывать исключения: — IndexOutOfRange (достигнут конец последовательности) Вычисление может не завершиться ввиду бесконечности последовательности	Порождает очередной элемент с помощью заданного правила и с учетом операций над последовательностью, если таковые требуется выполнить.
bool HasNext() const;	Возвращает, если ли следующий

² Т.е. очередь, головной элемент которой удаляется автоматически, если при добавлении в хвост количество элементов оказывается больше заданного порогового значения.

	элемент.
Optional<T> TryGetNext();	
Операции	
Generator <T>* Append(T item) const; Generator <T>* Append (Sequunce<T>* items) const; Generator <T>* Insert (T item) const; Generator <T>* Insert (Sequunce<T>* items) const; Generator <T>* Remove (T item) const; Generator <T>* Remove (Sequunce<T>* items) const;	

2. Потоки (streams)

Структуры с последовательным доступом иногда рассматривают через призму т.н. потоков, в которых чтение, как правило, рассматривается, логически или физически, как удаление элемента из последовательности (хотя бывают и исключения). Потоки во многом похожи на описанные выше ленивые последовательности, однако отличаются тем, что для потоков не определена алгебра (нет таких операций, как конкатенация, например). Как у всякой линейной структуры, у потока есть начало (начальный или первый элемент) и конец (последний или конечный элемент). Потоки как правило конечны, но нередко ведут себя как (конечные) ленивые последовательности (например, получение данных через сокет нередко рассматривается как чтение из потока).

Потоки бывают:

- только для чтения (получение начального элемента, который сразу же удаляется);
- только для записи (добавление элемента в конец);
- смешанный вариант.

Если в ленивых списках элементы задаются, по сути, процедурно, то в потоках они берутся из тех или иных источников – последовательностей различного типа, хранящихся в памяти, из файлов, из других потоков (например, потоков ввода-вывода).

Ниже приводится примерная внешняя спецификация потока только для чтения.

template <class T> class ReadOnlyStream	
Создание объекта	
ReadOnlyStream (Sequence<T>*); ReadOnlyStream (LazySequence<T>*); ReadOnlyStream (file, Deserializer<T>); ReadOnlyStream (string, Deserializer<T>); ReadOnlyStream (Stream);	Различные варианты создать поток: из какой-либо последовательности элементов, из файла или строки (причем необходимо указать также и способ десериализации объектов типа T из строки), из другого потока.

Декомпозиция	
bool IsEndOfStream () const;	Возвращает признак того, что достигнут конец потока и дальнейшее чтение невозможно.
T Read(); Может выбрасывать исключения: — EndOfStream (достигнут конец потока)	Считывает первый элемент потока и переходит на следующую позицию.
size_t GetPosition() const;	Возвращает текущую позицию в потоке (т.е. по сути – сколько элементов было считано на данный момент).
bool IsCanSeek() const;	Возвращает, возможно ли перемещение вдоль по потоку без чтения. Это возможно, например, если данные расположены в каком-либо sequence, иногда, – если в файле; невозможно – например, если данные читаются из сокета.
size_t Seek(size_t index)	Пробует перейти на заданную позицию, если возможно. Если позиция по какой-либо причине недостижима, может выбросить исключение; иногда – переходит на ближайшую возможную позицию и возвращает соответствующий индекс. Можно рассматривать как аналог операции Skip на списках.
bool IsCanGoBack() const;	Возвращает, возможно ли вернуться в ранее прочитанную позицию. При чтении файлов, в зависимости от того, в каком режиме открыт файл, это бывает возможным. Операция почти всегда имеет линейную сложность.
Операции	
void Open(); void Close();	После создания объекта могут потребоваться дополнительные действия, прежде чем станет возможным непосредственно чтение данных. Эти действия инкапсулируются в операцию Open. Аналогично, когда работа

	с потоком завершена, могут требоваться финализирующие действия, например, освобождение дескрипторов файлов. Для этого используется метод Close.
--	---

Ниже приводится примерная внешняя спецификация потока только для записи.

template <class T> class WriteOnlyStream	
Создание объекта	
WriteOnlyStream (Sequence<T>*); WriteOnlyStream (file, Deserializer<T>); WriteOnlyStream (Stream);	Различные варианты создать поток для записи: в какую-либо последовательность, в файл (причем необходимо указать также и способ сериализации объектов типа Т в строку), в другой поток.
Декомпозиция	
size_t GetPosition() const;	Возвращает текущую позицию в потоке (т.е. по сути – сколько элементов было всего записано в поток на данный момент).
Операции	
size_t Write (T);	Записывает элемент в конец потока и возвращает позицию, в которую будет осуществляться следующая запись. По сути, показывает, сколько элементов было всего записано в поток на данный момент.
void Open(); void Close();	После создания объекта могут требоваться дополнительные действия, прежде чем станет возможным непосредственно чтение данных. Эти действия инкапсулируются в операцию Open. Аналогично, когда работа с потоком завершена, могут требоваться финализирующие действия, например, освобождение дескрипторов файлов. Для этого используется метод Close.

Выбор варианта задания

№	Тема задания	Кол-во баллов
1.	Кодирование потока символов с буферизацией	
2.	Алгебра ленивых списков и алгебра ординальных чисел	
3.	Обработка потока событий (принятие решений)	
3.1.	Оценка вероятности событий, прогноз, выбор на основе оценки вероятности	
3.2.	Прогноз/экстраполяция с помощью конечно-разностных схем	
3.3.		
3.4.		
4.	Сбор статистики потока событий в режиме онлайн Сбор различных статистик, например, медианы, для бесконечных потоков/списков	
5.	Синхронизация потоков временных данных	
5.1.	Синхронизация данных по событиям	
5.2.	Выявление событий в не вполне синхронизированных потоках	
6.	Реализация машины/автомата состояний	
7.	Сжатие потока данных	
8.	Вычисление частоты появления подстрок в строке за 1 проход, из ленивого списка/потока	
9.	Построение алфавитного указателя (из потока/ленивого списка)	
10.	Сортировка потока данных	
10.1.	различными вариантами вставки (простой вставки, слиянием)	
10.2.	с помощью бинарной кучи / иного варианта очереди с приоритетами	
10.3.		
11.		
12.		
13.		
14.	Реализация «умного» генератора тестовых данных – обеспечивающего желаемые статистические или иные свойства порождаемых массивов данных	
15.	Поддержка множественных форматов для хранения данных (различных текстовых – csv, xml, json, yaml, и/или бинарных, в т.ч. собственной разработки).	
16.	Визуализация потоков (в т.ч. анимация)	

Описание вариантов заданий

1. Кодирование потока символов с буферизацией
2. Алгебра ленивых списков и алгебра ординалов
3. Обработка потока событий (принятие решений)

Постановка задачи: написать программный компонент, который обрабатывает поток событий, генерируя некоторую ответную реакцию на каждое событие. Источником событий считается подаваемая компоненту ленивая последовательность. Компонент необходимо

оснастить пользовательским интерфейсом, визуализирующим как поступающий поток событий, так и ответ компонента.

3.1. на основе правил

Ответная реакция задается с помощью правил. Правила могут учитывать как только последнее поступившее событие, так и несколько предшествующих. Должен быть реализовать редактор правил.

3.2. на вероятностной основе

На каждое поступающее событие компонент: (1) строит прогноз, какое будет следующее событие исходя из оценки вероятности событий, (2) .

3.3. прогноз/коррекция

4. Сбор статистики в режиме онлайн

5. Синхронизация потоков временных данных

5.1. Синхронизация данных по событиям

Потоком временных данных в данной задаче называется потенциально бесконечная последовательность пар «время-значение». Т.е. такой поток моделирует поступающие с какого-либо датчика или измерительного устройства данные, помеченные временем снятия показаний. При этом снятие данных, описывающих отдельное событие или целостное состояние системы не происходит одномоментно, а в течение некоторого интервала τ . Причем есть вероятность, что отдельные датчики по какой-либо причине могут не выдать измеренные значения.

Например, работа системы фиксируется 3-мя устройствами, каждое из которых представлено отдельным потоком. Пусть в момент t происходит событие. До наступления момента времени $t + \tau$ все датчики должны зафиксировать свои значения и поместить их в соответствующую последовательность. Если какой-то датчик этого не сделал, его показания считаются отсутствующими и не помещаются в соответствующую последовательность.

Те значения, которые были зафиксированы, характеризуются величинами Δt_i – отставаниями от фактического момента события: $t_{i_1} = t + \Delta t_{i_1}, \dots, t_{i_n} = t + \Delta t_{i_n}$. Если t не известно, то, зная τ и считая, что величины $t_{i_1}, \dots, t_{i_n}$ равномерно распределены на интервале $[t; t + \Delta t]$, величину t с некоторой погрешностью можно оценить по формуле:

$$t \cong \frac{t_{i_1} + \dots + t_{i_n}}{n} - \frac{\tau}{2}$$

Задача состоит в том, что имея на входе n ленивых последовательностей, содержащий такие не совсем синхронизированные временные ряды, построить новую последовательность из $(n + 1)$ -арных кортежей, в которых первым элементом будет время вычисленное время t , а остальное – значения показаний (или Option::None).

Для оценки точности полученного решения необходимо провести серию из экспериментов, в каждом из которых наряду с n последовательностями показаний используется дополнительная последовательность, содержащая «эталонные» значения t . Отметим, что на практике во многих технических системах имеется специальное устройство (триггер), срабатывание которого как раз и запускает считывание показаний с устройств. В некоторых сложных комплексах таких устройств может быть несколько.

5.2. Выявление событий в не вполне синхронизированных потоках

Предположим, что в предыдущей задаче показания, не попавшие в интервал $[t; t + \tau]$ не отбрасываются, а попадают в последовательность. Такая последовательность становится замусоренной, в ней значения могут быть не только пропущены, но и присутствовать лишние. Основная задача остается прежней: идентифицировать события и построить последовательность $(n + 1)$ -арных кортежей, включающих оценку времени и описывающих эти события.

6. Реализация машины состояний

7. Реализация машины Тьюринга

8. Реализация машины Маркова

9. Сжатие потока дан

10. Построение алфавитного указателя

Критерии оценки

1.	Качество программного кода:	– стиль (в т.ч.: имена, отступы и проч.) (0-2) – структурированность (напр. декомпозиция сложных функций на более простые) (0-2) – качество основных и второстепенных алгоритмов (напр. обработка граничных случаев и некорректных исходных данных и т.п.) (0-2)	0-6 баллов
2.	Качество пользовательского интерфейса:	– предоставляемые им возможности (0-2) – наличие ручного/автоматического ввода исходных данных (0-2) – настройка параметров для автоматического режима – отображение исходных данных и промежуточных и конечных результатов и др. (0-2)	0-6 баллов
3.	Качество тестов	– степень покрытия – читаемость – качество проверки (граничные и некорректные значения, и др.)	0-3 баллов
Дополнительно			
4.	Владение теорией	знание алгоритмов, области их применимости, умение сравнивать с аналогами, оценить сложность, корректность реализации	0-3 баллов
5.	Оригинальность реализации	оцениваются отличительные особенности конкретной реализации – например, общность структур данных, наличие продвинутых графических	0-5 баллов

		средств, средств ввода-вывода, интеграции с внешними системами и др.	
		Итого	0-15(23) баллов

Для получения зачета за выполнения лабораторной работы необходимо соблюдение всех перечисленных условий:

- оценка за п. 1 должна быть не менее 3 баллов
- оценка за п. 2 должна быть не менее 3 баллов
- оценка за п. 3 должна быть больше 0